

AI ASSISTED CODING

ASSIGNMENT-13

NAME: Geethanjali

HTNO: 2303A51005

Task Description #1 (Refactoring – Removing Code Duplication)

- Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.

- Instructions:

- o Prompt AI to identify duplicate logic and replace it with functions or classes.

- o Ensure the refactored code maintains the same output.

- o Add docstrings to all functions.

- Sample Legacy Code:

```
# Legacy script with repeated logic
```

```
print("Area of Rectangle:", 5 * 10)
```

```
print("Perimeter of Rectangle:", 2 * (5 + 10))
```

```
print("Area of Rectangle:", 7 * 12)
```

```
print("Perimeter of Rectangle:", 2 * (7 + 12))
```

```
print("Area of Rectangle:", 10 * 15)
```

```
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

- Expected Output:

- o Refactored code with a reusable function and no duplication.

- o Well documented code

```
Rectangle.py > ...
1 def calculate_rectangle_metrics(length: float, width: float) -> tuple[float, float]:
2     """Compute area and perimeter for a rectangle.
3
4     Parameters:
5         length (float): length of the rectangle
6         width (float): width of the rectangle
7
8     Returns:
9         tuple[float, float]: (area, perimeter)
10
11     The formulas used are::
12         area = length * width
13         perimeter = 2 * (length + width)
14     """
15     area = length * width
16     perimeter = 2 * (length + width)
17     return area, perimeter
18
19
20 def print_rectangle_metrics(length: float, width: float) -> None:
21     """Prints the area and perimeter for a rectangle with given dimensions.
22
23     This helper wraps :func:`calculate_rectangle_metrics` and formats the output
24     in a human-readable way.
25
26     Parameters:
27         length (float): length of the rectangle
28         width (float): width of the rectangle
29     """
30     area, perimeter = calculate_rectangle_metrics(length, width)
31     print(f"Area of rectangle (length={length}, width={width}): {area}")
32     print(f"Perimeter of rectangle (length={length}, width={width}): {perimeter}")
33
34
35 # Example usage for several rectangles. If the module is executed as a script,
36 # the following code will run. In a larger program it would typically be
37 # imported and the functions called directly.
38 if __name__ == "__main__":
39     rectangles = [(5, 10), (7, 12), (10, 15)]
40
41     for length, width in rectangles:
42         print_rectangle_metrics(length, width)
43
44
45 perimeter = 2 * (length + width)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
zsh
/usr/local/bin/python3 "/Users/geethanjali/Downloads/ Rectangle.py"
geethanjali@Geethanjali Downloads % /usr/local/bin/python3 "/Users/geethanjali/Downloads/ Rectangle.py"
Area of rectangle (length=5, width=10): 50
Perimeter of rectangle (length=5, width=10): 30
Area of rectangle (length=7, width=12): 84
Perimeter of rectangle (length=7, width=12): 38
Area of rectangle (length=10, width=15): 150
Perimeter of rectangle (length=10, width=15): 50
geethanjali@Geethanjali Downloads % /usr/local/bin/python3 "/Users/geethanjali/Downloads/ Rectangle.py"
Area of rectangle (length=5, width=10): 50
Perimeter of rectangle (length=5, width=10): 30
Area of rectangle (length=7, width=12): 84
Perimeter of rectangle (length=7, width=12): 38
Area of rectangle (length=10, width=15): 150
Perimeter of rectangle (length=10, width=15): 50
geethanjali@Geethanjali Downloads %
```

Task Description #2 (Refactoring – Extracting Reusable Functions)

- Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.

- Instructions:

- o Identify repeated or related logic and extract it into reusable functions.

- o Ensure the refactored code is modular, easy to read, and documented with docstrings.

- Sample Legacy Code:

Docstring style gives a clear description, arguments, and return values. It is readable but may lack strict structure for larger security projects.

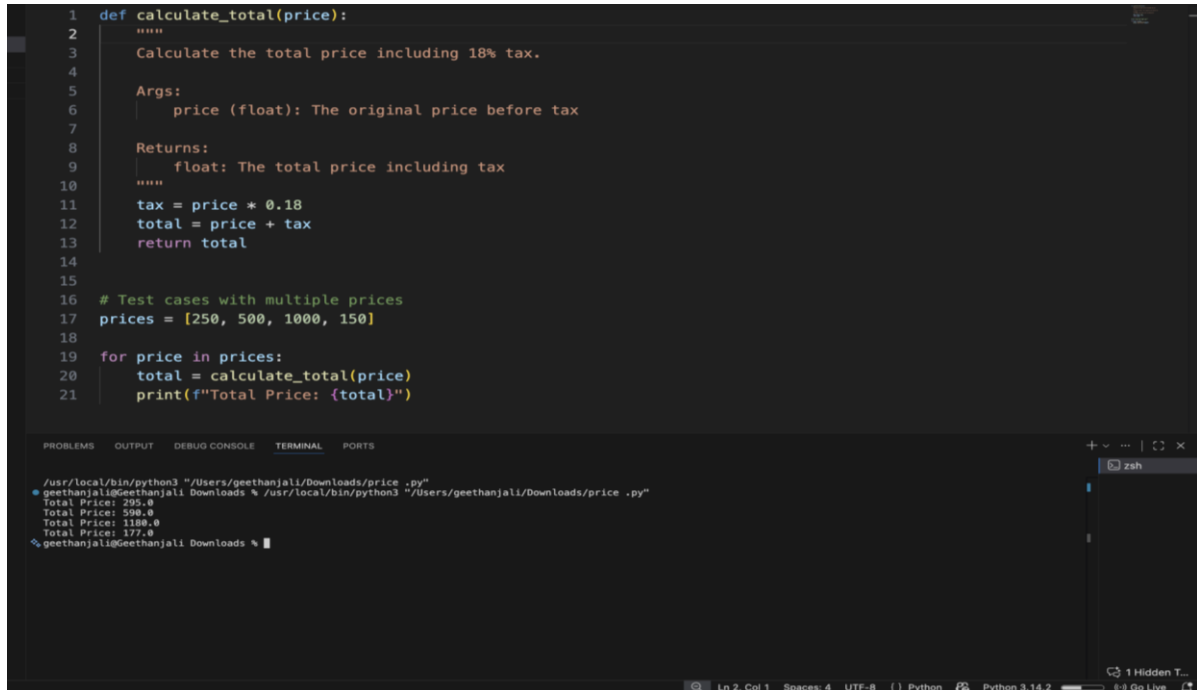
Inline documentation is short and simple, but not detailed enough for security-related code where clarity is very important.

Google style documentation is more structured and detailed. It clearly explains arguments, return types, and possible errors which is important in security code.

For security-related code, Google style documentation is the most appropriate because it is clear, structured, and explains exceptions properly, reducing misunderstanding and security risks.

Legacy script with inline repeated logic

price = 250 tax = price * 0.18 total = price + tax print("Total Price:", total) price = 500 tax = price * 0.18 total = price + tax print("Total Price:", total) • Expected Output: o Code with a function calculate_total(price) that can be reused for multiple price inputs. o Well documented code



```
1 def calculate_total(price):
2     """
3     Calculate the total price including 18% tax.
4
5     Args:
6     price (float): The original price before tax
7
8     Returns:
9     float: The total price including tax
10    """
11    tax = price * 0.18
12    total = price + tax
13    return total
14
15
16 # Test cases with multiple prices
17 prices = [250, 500, 1000, 150]
18
19 for price in prices:
20     total = calculate_total(price)
21     print(f"Total Price: {total}")
```

Terminal Output:

```
/usr/local/bin/python3 "/Users/geethanjali/Downloads/price.py"
geethanjali@geethanjali Downloads % /usr/local/bin/python3 "/Users/geethanjali/Downloads/price.py"
Total Price: 295.0
Total Price: 590.0
Total Price: 1180.0
Total Price: 177.0
geethanjali@geethanjali Downloads %
```

ask Description #3: Refactoring Using Classes and Methods (Eliminating Redundant Conditional Logic)

Refactor a Python script that contains repeated if–elif–else grading logic by implementing a structured, object-oriented solution using a class and a method.

Problem Statement

The given script contains duplicated conditional statements used to assign grades based on student marks. This redundancy violates clean code principles and reduces maintainability.

You are required to refactor the script using a class-based design to improve modularity, reusability, and readability while preserving the original grading logic.

Mandatory Implementation Requirements

1. Class Name: GradeCalculator

2. Method Name: calculate_grade(self, marks)

3. The method must:

- o Accept marks as a parameter

- o Return the corresponding grade as a string.

- o The grading logic must strictly follow the conditions below:

- Marks ≥ 90 and $\leq 100 \rightarrow$ "Grade A"

- Marks $\geq 80 \rightarrow$ "Grade B"

- Marks $\geq 70 \rightarrow$ "Grade C"

- Marks $\geq 40 \rightarrow$ "Grade D"

- Marks $\geq 0 \rightarrow$ "Fail"

Note: Assume marks are within the valid range of 0 to 100.

4. Include proper docstrings for:

- o The class

- o The method (with parameter and return descriptions)

5. The method must be reusable and called multiple times without rewriting conditional logic.

- Given code:

```
marks = 85
```

```
if marks >= 90:
```

```
    print("Grade A")
```

```
elif marks >= 75:
```

```
    print("Grade B")
```

```
else:
```

```
    print("Grade C")
```

```
marks = 72
```

```
if marks >= 90:
```

```
    print("Grade A")
```

```
elif marks >= 75:
```

```
print("Grade B")
```

```
else:
```

```
print("Grade C")
```

Expected Output:

- Define a class named GradeCalculator.
- Implement a method `calculate_grade(self, marks)` inside the class.

Create an object of the class.

- Call the method for different student marks.
- Print the returned grade values.

```

class Grade.py >
1 class GradeCalculator:
2     """
3     A class to calculate student grades based on marks.
4
5     This class provides functionality to determine letter grades
6     from numeric marks using a predefined grading scale.
7     """
8
9     def calculate_grade(self, marks):
10        """
11        Calculate the letter grade for the given marks.
12
13        Parameters:
14            marks (int/float): The student's marks (0-100)
15
16        Returns:
17            str: The corresponding grade as a string
18        """
19        if marks >= 90 and marks <= 100:
20            return "Grade A"
21        elif marks >= 80:
22            return "Grade B"
23        elif marks >= 70:
24            return "Grade C"
25        elif marks >= 40:
26            return "Grade D"
27        else:
28            return "Fail"
29
30
31 # Create an object of the class
32 calculator = GradeCalculator()
33
34 # Call the method for different student marks
35 marks1 = 85
36 print(f"Marks: {marks1}, {calculator.calculate_grade(marks1)}")
37
38 marks2 = 72
39 print(f"Marks: {marks2}, {calculator.calculate_grade(marks2)}")
40
41 # Additional test cases
42 marks3 = 95
43 print(f"Marks: {marks3}, {calculator.calculate_grade(marks3)}")

```

```

29
30
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
+ - | | x
zsh
/usr/local/bin/python3 ~/Users/geethanjali/Downloads/class Grade.py
geethanjali@geethanjali Downloads % /usr/local/bin/python3 ~/Users/geethanjali/Downloads/class Grade.py
Marks: 85, Grade B
Marks: 72, Grade C
Marks: 95, Grade A
Marks: 58, Grade D
Marks: 35, Fail
geethanjali@geethanjali Downloads % /usr/local/bin/python3 ~/Users/geethanjali/Downloads/class Grade.py
Marks: 85, Grade B
Marks: 72, Grade C
Marks: 95, Grade A
Marks: 58, Grade D
Marks: 35, Fail
geethanjali@geethanjali Downloads %

```

Task Description #4 (Refactoring – Converting Procedural Code to Functions)

- Task: Use AI to refactor procedural input–processing logic into functions.

Instructions:

- o Identify input, processing, and output sections.
- o Convert each into a separate function.
- o Improve code readability without changing behavior.

- Sample Legacy Code:

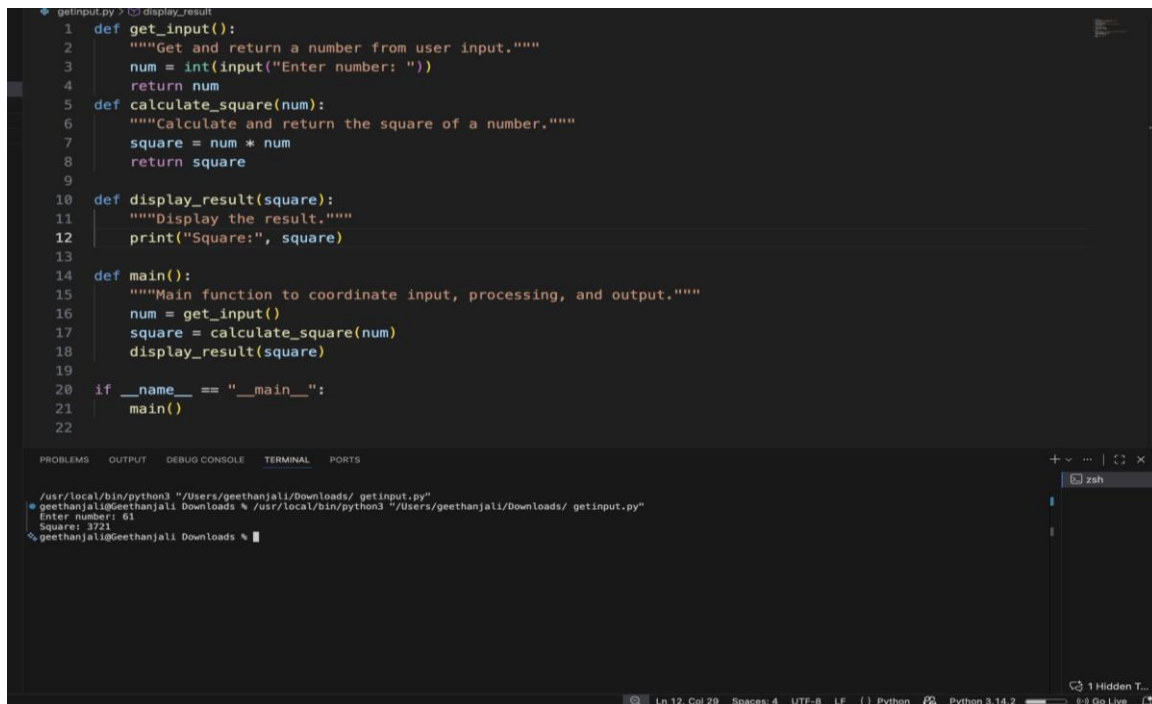
```
num = int(input("Enter number: "))
```

```
square = num * num
```

```
print("Square:", square)
```

- Expected Output:

- o Modular code using functions like `get_input()`, `calculate_square()`, and `display_result()`.



```
1 def get_input():
2     """Get and return a number from user input."""
3     num = int(input("Enter number: "))
4     return num
5
6 def calculate_square(num):
7     """Calculate and return the square of a number."""
8     square = num * num
9     return square
10
11 def display_result(square):
12     """Display the result."""
13     print("Square:", square)
14
15 def main():
16     """Main function to coordinate input, processing, and output."""
17     num = get_input()
18     square = calculate_square(num)
19     display_result(square)
20
21 if __name__ == "__main__":
22     main()
```

Terminal Output:

```
/usr/local/bin/python3 /Users/geethanjali/Downloads/ getinput.py
geethanjali@Geethanjali Downloads % /usr/local/bin/python3 /Users/geethanjali/Downloads/ getinput.py
Enter number: 61
Square: 3721
geethanjali@Geethanjali Downloads %
```

Task 5 (Refactoring Procedural Code into OOP Design)

- Task: Use AI to refactor procedural code into a class-based design.

Focus Areas:

- o Object-Oriented principles

- o Encapsulation

Legacy Code:

```
salary = 50000
```

```
tax = salary * 0.2
```

```
net = salary - tax
```

```
print(net)
```

Expected Outcome:

- o A class like `EmployeeSalaryCalculator` with methods and attributes.

```
class EmployeeSalaryCalculator:
    """Encapsulates employee salary calculations using OOP principles."""

    TAX_RATE = 0.2

    def __init__(self, salary):
        """Initialize with employee salary.

        Args:
            salary (float): Annual salary in dollars.
        """
        self._salary = salary
        self._tax = None
        self._net_salary = None

    @property
    def salary(self):
        """Get the gross salary."""
        return self._salary

    @salary.setter
    def salary(self, value):
        """Set the gross salary.

        If value < 0:
            raise ValueError("Salary cannot be negative")
        """
        self._salary = value
        self._tax = None
        self._net_salary = None

    def calculate_tax(self):
        """Calculate tax based on salary and tax rate.

        Returns:
            float: Tax amount.
        """
        if self._tax is None:
            self._tax = self._salary * self.TAX_RATE
        return self._tax

    def calculate_net_salary(self):
        """Calculate net salary after tax deduction.

        Returns:
            float: Net salary.
        """
        if self._net_salary is None:
            self._net_salary = self._salary - self.calculate_tax()
        return self._net_salary

    def display_salary_breakdown(self):
        """Display salary breakdown.

        print(f"Gross Salary: ${self._salary:,.2f}")
        print(f"Tax {(self.TAX_RATE * 100)%}: ${self.calculate_tax():,.2f}")
        print(f"Net Salary: ${self.calculate_net_salary():,.2f}")
        """

# Usage
if __name__ == "__main__":
    calculator = EmployeeSalaryCalculator(50000)
    calculator.display_salary_breakdown()
```

```
zsh
/usr/local/bin/python3 "/Users/geethanjali/Downloads/EmployeeSalary.py"
geethanjali@geethanjali Downloads % /usr/local/bin/python3 "/Users/geethanjali/Downloads/EmployeeSalary.py"
Gross Salary: $50,000.00
Tax (20.0%): $10,000.00
Net Salary: $40,000.00
geethanjali@geethanjali Downloads % /usr/local/bin/python3 "/Users/geethanjali/Downloads/EmployeeSalary.py"
Gross Salary: $50,000.00
Tax (20.0%): $10,000.00
Net Salary: $40,000.00
geethanjali@geethanjali Downloads %
```

Task 6 (Optimizing Search Logic)

- Task: Refactor inefficient linear searches using appropriate data structures.

- Focus Areas:

- o Time complexity

- o Data structure choice

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]
```

```
name = input("Enter username: ")
```

```
found = False
```

```
for u in users:
```

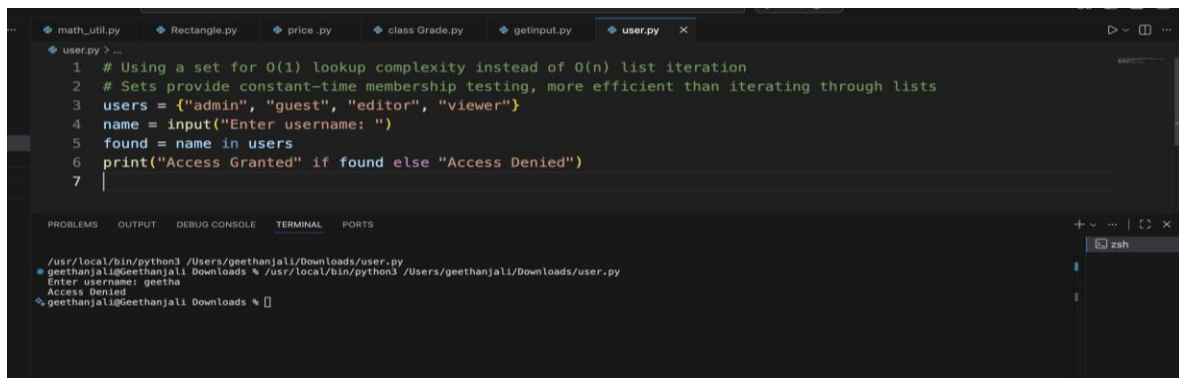
```
    if u == name:
```

```
        found = True
```

```
print("Access Granted" if found else "Access Denied")
```

Expected Outcome:

o Use of sets or dictionaries with complexity justification



```
1 # Using a set for O(1) lookup complexity instead of O(n) list iteration
2 # Sets provide constant-time membership testing, more efficient than iterating through lists
3 users = {"admin", "guest", "editor", "viewer"}
4 name = input("Enter username: ")
5 found = name in users
6 print("Access Granted" if found else "Access Denied")
7
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
/usr/local/bin/python3 /Users/geethanjali/Downloads/user.py
geethanjali@geethanjali Downloads % /usr/local/bin/python3 /Users/geethanjali/Downloads/user.py
Enter username: geetha
Access Denied
geethanjali@geethanjali Downloads %
```

Task 7 – Refactoring the Library Management System

Problem Statement

You are provided with a poorly structured Library Management script that:

- Contains repeated conditional logic
- Does not use reusable functions
- Lacks documentation
- Uses print-based procedural execution
- Does not follow modular programming principles

Your task is to refactor the code into a proper format

1. Create a module library.py with functions:

o add_book(title, author, isbn)

o remove_book(isbn)

o search_book(isbn)

2. Insert triple quotes under each function and let Copilot complete the docstrings.

3. Generate documentation in the terminal.
4. Export the documentation in HTML format.
5. Open the file in a browser. Given Code

Library Management System (Unstructured Version)

This code needs refactoring into a proper module with documentation.

```
library_db = {}
```

Adding first book

```
title = "Python Basics" author = "John Doe" isbn = "101" if isbn not in library_db:
library_db[isbn] = {"title": title, "author": author} print("Book added successfully.") else:
print("Book already exists.")
```

Adding second book (duplicate logic)

```
title = "AI Fundamentals" author = "Jane Smith" isbn = "102" if isbn not in library_db:  
library_db[isbn] = {"title": title, "author": author} print("Book added successfully.") else:  
print("Book already exists.")
```

Searching book (repeated logic structure)

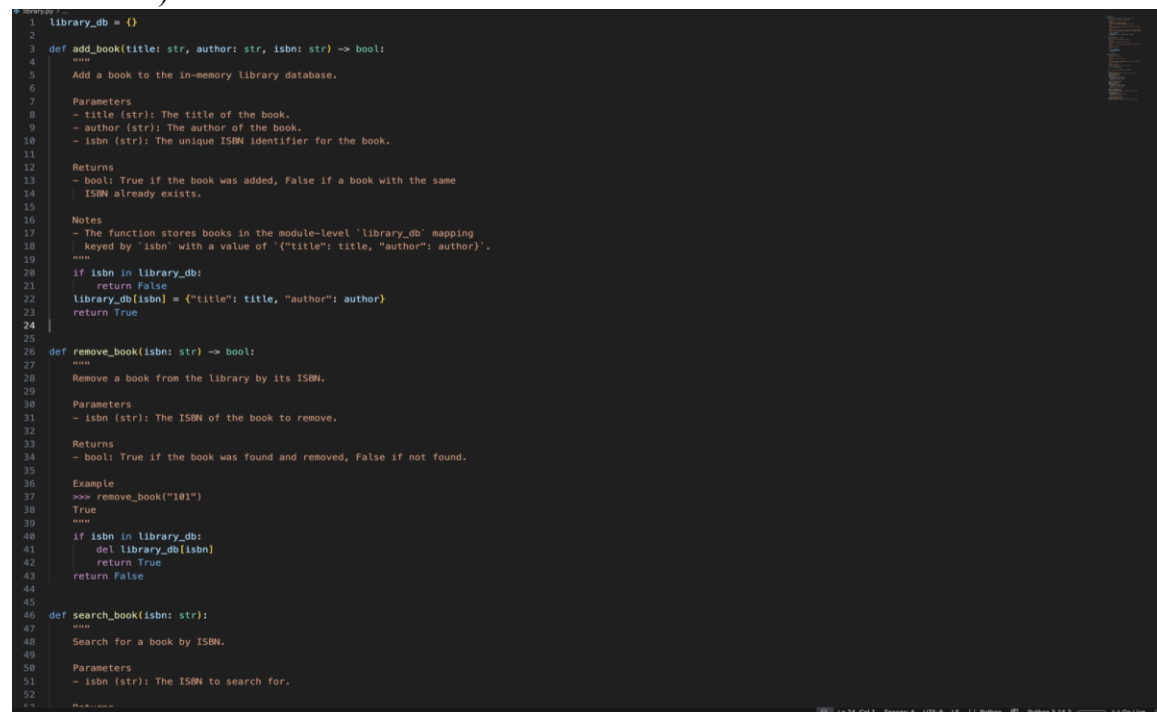
```
isbn = "101" if isbn in library_db: print("Book Found:", library_db[isbn]) else: print("Book  
not found.")
```

Removing book (again repeated pattern)

```
isbn = "101" if isbn in library_db: del library_db[isbn] print("Book removed successfully.")  
else: print("Book not found.")
```

Searching again

```
isbn = "101" if isbn in library_db: print("Book Found:", library_db[isbn]) else: print("Book  
not found.")
```



```
1 library_db = {}  
2  
3 def add_book(title: str, author: str, isbn: str) -> bool:  
4     """  
5     Add a book to the in-memory library database.  
6  
7     Parameters  
8     ~ title (str): The title of the book.  
9     ~ author (str): The author of the book.  
10    ~ isbn (str): The unique ISBN identifier for the book.  
11  
12    Returns  
13    ~ bool: True if the book was added, False if a book with the same  
14    ~ ISBN already exists.  
15  
16    Notes  
17    ~ The function stores books in the module-level 'library_db' mapping  
18    ~ keyed by 'isbn' with a value of '{"title": title, "author": author}'.  
19    """  
20    if isbn in library_db:  
21        return False  
22    library_db[isbn] = {"title": title, "author": author}  
23    return True  
24  
25  
26 def remove_book(isbn: str) -> bool:  
27     """  
28     Remove a book from the library by its ISBN.  
29  
30     Parameters  
31     ~ isbn (str): The ISBN of the book to remove.  
32  
33     Returns  
34     ~ bool: True if the book was found and removed, False if not found.  
35  
36     Example  
37     >>> remove_book("101")  
38     True  
39     """  
40    if isbn in library_db:  
41        del library_db[isbn]  
42        return True  
43    return False  
44  
45  
46 def search_book(isbn: str):  
47     """  
48     Search for a book by ISBN.  
49  
50     Parameters  
51     ~ isbn (str): The ISBN to search for.  
52  
53     Returns  
54     ~ dict: A dictionary containing the book's title and author, or None if not found.  
55     """  
56    if isbn in library_db:  
57        return library_db[isbn]  
58    return None  
59  
60  
61 if __name__ == "__main__":  
62     # Add books  
63     add_book("AI Fundamentals", "Jane Smith", "102")  
64     add_book("Data Science", "John Doe", "103")  
65  
66     # Search for a book  
67     isbn = "101"  
68     result = search_book(isbn)  
69     if result:  
70         print(f"Book Found: {result}")  
71     else:  
72         print("Book not found.")  
73  
74     # Remove a book  
75     isbn = "101"  
76     result = remove_book(isbn)  
77     if result:  
78         print(f"Book removed successfully.")  
79     else:  
80         print("Book not found.")  
81  
82     # Search again  
83     isbn = "101"  
84     result = search_book(isbn)  
85     if result:  
86         print(f"Book Found: {result}")  
87     else:  
88         print("Book not found.")
```

```
library.py
46 def search_book(isbn: str):
47     """dict or None: Returns a dictionary with keys 'title' and 'author'
48     if found, otherwise 'None'."""
49
50     Example
51     >>> search_book("101")
52     {'title': 'Python Basics', 'author': 'John Doe'}
53     """
54     return library_db.get(isbn)
55
56
57 __all__ = ["add_book", "remove_book", "search_book"]
58
59
60 if __name__ == "__main__":
61     # Demo: refactored use of the previously unstructured code
62     print("Adding first book")
63     title = "Python Basics"
64     author = "John Doe"
65     isbn = "101"
66     if add_book(title, author, isbn):
67         print("Book added successfully.")
68     else:
69         print("Book already exists.")
70
71     print("Adding second book")
72     title = "AI Fundamentals"
73     author = "Jane Smith"
74     isbn = "102"
75     if add_book(title, author, isbn):
76         print("Book added successfully.")
77     else:
78         print("Book already exists.")
79
80     print("Searching book 101")
81     result = search_book("101")
82     print("Book Found:" if result else "Book not found.", result)
83
84     print("Removing book 101")
85     if remove_book("101"):
86         print("Book removed successfully.")
87     else:
88         print("Book not found.")
89
90     print("Searching again for 101")
91     result = search_book("101")
92     print("Book Found:" if result else "Book not found.", result)
93
94
95
96
97
98
99
100
```

Demo: refactored use of the previously unstructured code

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

/usr/local/bin/python3 /Users/geethanjali/Desktop/AI/library.py
● geethanjali@Geethanjali AI % /usr/local/bin/python3 /Users/geethanjali/Desktop/AI/library.py
Adding first book
Book added successfully.
Adding second book
Book added successfully.
Searching book 101
Book Found: {'title': 'Python Basics', 'author': 'John Doe'}
Removing book 101
Book removed successfully.
Searching again for 101
Book not found. None
geethanjali@Geethanjali AI %

Task 8– Fibonacci Generator

Write a program to generate Fibonacci series up to n.

The initial code has:

- Global variables.
- Inefficient loop.
- No functions or modularity.

Task for Students:

- Refactor into a clean reusable function (generate_fibonacci).
- Add docstrings and test cases.
- Compare AI-refactored vs original.

Bad Code Version:

```
# fibonacci bad version

n=int(input("Enter limit: "))

a=0

b=1

print(a)

print(b)

for i in range(2,n):

    c=a+b

    print(c)

    a=b

    b=c
```

```
def generate_fibonacci(n):
    """
    Generate Fibonacci series up to n terms.

    Parameters:
    n (int): Number of terms to generate

    Returns:
    list: A list containing Fibonacci series up to n terms
    """
    if n <= 0:
        return []
    elif n == 1:
        return [0]

    fibonacci_series = [0, 1]

    for _ in range(2, n):
        next_value = fibonacci_series[-1] + fibonacci_series[-2]
        fibonacci_series.append(next_value)

    return fibonacci_series

# Test Cases
if __name__ == "__main__":
    print("Test Case 1 (n=1):", generate_fibonacci(1))
    print("Test Case 2 (n=5):", generate_fibonacci(5))
    print("Test Case 3 (n=10):", generate_fibonacci(10))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
/usr/local/bin/python3 "/Users/geethanjali/Desktop/AI/def generate_fibonacci(n):.py"
geethanjali@Geethanjali AI % /usr/local/bin/python3 "/Users/geethanjali/Desktop/AI/def generate_fibonacci(n):.py"
Test Case 1 (n=1): [0]
Test Case 2 (n=5): [0, 1, 1, 2, 3]
Test Case 3 (n=10): [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
geethanjali@Geethanjali AI % /usr/local/bin/python3 "/Users/geethanjali/Desktop/AI/def generate_fibonacci(n):.py"
Test Case 1 (n=1): [0]
Test Case 2 (n=5): [0, 1, 1, 2, 3]
Test Case 3 (n=10): [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
geethanjali@Geethanjali AI %
```

Task 9 – Twin Primes Checker

Twin primes are pairs of primes that differ by 2 (e.g., 11 and 13, 17 and 19).

The initial code has:

- Inefficient prime checking.
- No functions.
- Hardcoded inputs.

Task for Students:

- Refactor into `is_prime(n)` and `is_twin_prime(p1, p2)`.
- Add docstrings and optimize.
- Generate a list of twin primes in a given range using AI.

Bad Code Version:

```
# twin primes bad version
```

```

a=11
b=13
fa=0
for i in range(2,a):
    if a%i==0:
        fa=1
fb=0
for i in range(2,b):
    if b%i==0:
        fb=1
if fa==0 and fb==0 and abs(a-b)==2:
    print("Twin Primes")
else:
    print("Not Twin Primes")

```

```

1  import math
2
3  def is_prime(n):
4      """
5      Check whether a number is prime.
6
7      Parameters:
8      n (int): Number to check
9
10     Returns:
11     bool: True if n is prime, False otherwise
12     """
13     if n <= 1:
14         return False
15     if n <= 3:
16         return True
17     if n % 2 == 0:
18         return False
19
20     # Check divisibility up to sqrt(n)
21     for i in range(3, int(math.sqrt(n)) + 1, 2):
22         if n % i == 0:
23             return False
24     return True
25
26
27 def is_twin_prime(p1, p2):
28     """
29     Check whether two numbers are twin primes.
30
31     Parameters:
32     p1 (int): First number
33     p2 (int): Second number
34
35     Returns:
36     bool: True if both are prime and differ by 2

```

Task 10 – Refactoring the Chinese Zodiac Program

Objective

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

The current program reads a year from the user and prints the corresponding

Chinese Zodiac sign. However, the implementation contains repetitive conditional logic, lacks modular design, and does not follow clean coding principles.

Your task is to refactor the code to improve readability, maintainability, and structure.

Chinese Zodiac Cycle (Repeats Every 12 Years)

1. Rat
2. Ox
3. Tiger
4. Rabbit
5. Dragon
6. Snake
7. Horse
8. Goat (Sheep)
9. Monkey
10. Rooster
11. Dog
12. Pig

Chinese Zodiac Program (Unstructured Version)

This code needs refactoring.

```
year = int(input("Enter a year: "))
```

```
if year % 12 == 0:
```

```
    print("Monkey")
```

```
elif year % 12 == 1:
```

```
    print("Rooster")
```

```
elif year % 12 == 2:
```

```
    print("Dog")
```

```
elif year % 12 == 3:
```

```
    print("Pig")
```

```
elif year % 12 == 4:
```

```
    print("Rat")
```

```
elif year % 12 == 5:
```

```
    print("Ox")
```

```
elif year % 12 == 6:
```

```
    print("Tiger")
```

```
elif year % 12 == 7:
```

```
    print("Rabbit")
```

```
elif year % 12 == 8:
```

```
    print("Dragon")
```

```
elif year % 12 == 9:
```

```
    print("Snake")
```

```
elif year % 12 == 10:
```

```
    print("Horse")
```

```
elif year % 12 == 11:
```

```
    print("Goat")
```

You must:

1. Create a reusable function: `get_zodiac(year)`
2. Replace the if-elif chain with a cleaner structure (e.g., list or dictionary).
3. Add proper docstrings.
4. Separate input handling from logic.
5. Improve readability and maintainability.
6. Ensure output remains correct.

```

def get_zodiac(year):
    """
    Determine the Chinese Zodiac sign for a given year.

    Parameters:
    year (int): The year for which the zodiac sign is required

    Returns:
    str: The corresponding Chinese Zodiac sign
    """
    zodiac_cycle = [
        "Monkey", # 0
        "Rooster", # 1
        "Dog", # 2
        "Pig", # 3
        "Rat", # 4
        "Ox", # 5
        "Tiger", # 6
        "Rabbit", # 7
        "Dragon", # 8
        "Snake", # 9
        "Horse", # 10
        "Goat" # 11
    ]

    return zodiac_cycle[year % 12]

def main():
    """
    Handle user input and display the zodiac sign.
    """
    try:
        year = int(input("Enter a year: "))
        sign = get_zodiac(year)
        print(f"Chinese Zodiac Sign: {sign}")
    except ValueError:
        print("Invalid input. Please enter a valid integer year.")

# Program execution starts here
if __name__ == "__main__":
    main()

```

```

/usr/local/bin/python3 /Users/geethanjali/Desktop/AI/def_get_zodiac(year):.py
geethanjali@Geethanjali AI % /usr/local/bin/python3 /Users/geethanjali/Desktop/AI/def_get_zodiac(year):.py
Enter a year: 2005
Chinese Zodiac Sign: Rooster
geethanjali@Geethanjali AI %

```

Task 11 – Refactoring the Harshad (Niven) Number Checker

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

A Harshad (Niven) number is a number that is divisible by the sum of its digits.

For example:

- $18 \rightarrow 1 + 8 = 9 \rightarrow 18 \div 9 = 2$ (Harshad Number)
- $19 \rightarrow 1 + 9 = 10 \rightarrow 19 \div 10 \neq \text{integer}$ (Not Harshad)

Problem Statement

The current implementation:

- Mixes logic and input handling
- Uses redundant variables
- Does not use reusable functions properly
- Returns print statements instead of boolean values

- Lacks documentation

You must refactor the code to follow clean coding principles.

Harshad Number Checker (Unstructured Version)

```
num = int(input("Enter a number: "))
```

```
temp = num
```

```
sum_digits = 0
```

```
while temp > 0:
```

```
    digit = temp % 10
```

```
    sum_digits = sum_digits + digit
```

```
    temp = temp // 10
```

```
if sum_digits != 0:
```

```
    if num % sum_digits == 0:
```

```
        print("True")
```

```
    else:
```

```
        print("False")
```

```
else:
```

```
    print("False")
```

You must:

1. Create a reusable function: `is_harshad(number)`
2. The function must:
 - o Accept an integer parameter.
 - o Return True if the number is divisible by the sum of its digits.
 - o Return False otherwise.
3. Separate user input from core logic.
4. Add proper docstrings.
5. Improve readability and maintainability.
6. Ensure the program handles edge cases (e.g., 0, negative numbers).

```
def is_harshad(number):
    """
    Calculate sum of digits
    """
    digit_sum = sum(int(digit) for digit in str(number))

    # Avoid division by zero (extra safety)
    if digit_sum == 0:
        return False

    return number % digit_sum == 0

def main():
    """
    Handle user input and display result.
    """
    try:
        num = int(input("Enter a number: "))
        result = is_harshad(num)
        print(result)
    except ValueError:
        print("Invalid input. Please enter a valid integer.")

# Program execution starts here
if __name__ == "__main__":
    main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
/usr/local/bin/python3 "/Users/geethanjali/Desktop/AI/def_is_harshad(number).py"
geethanjali@geethanjali AI % /usr/local/bin/python3 "/Users/geethanjali/Desktop/AI/def_is_harshad(number).py"
Enter a number: 09
True
geethanjali@geethanjali AI %
```

Spaces: 4 UTF-8 Python Python 3.14.2 8:4 Go Live

Task 12 – Refactoring the Factorial Trailing Zeros Program

Refactor the given poorly structured Python script into a clean, modular, and efficient implementation.

The program calculates the number of trailing zeros in $n!$ (factorial of n).

Problem Statement

The current implementation:

- Calculates the full factorial (inefficient for large n)
- Mixes input handling with business logic
- Uses print statements instead of return values
- Lacks modular structure and documentation

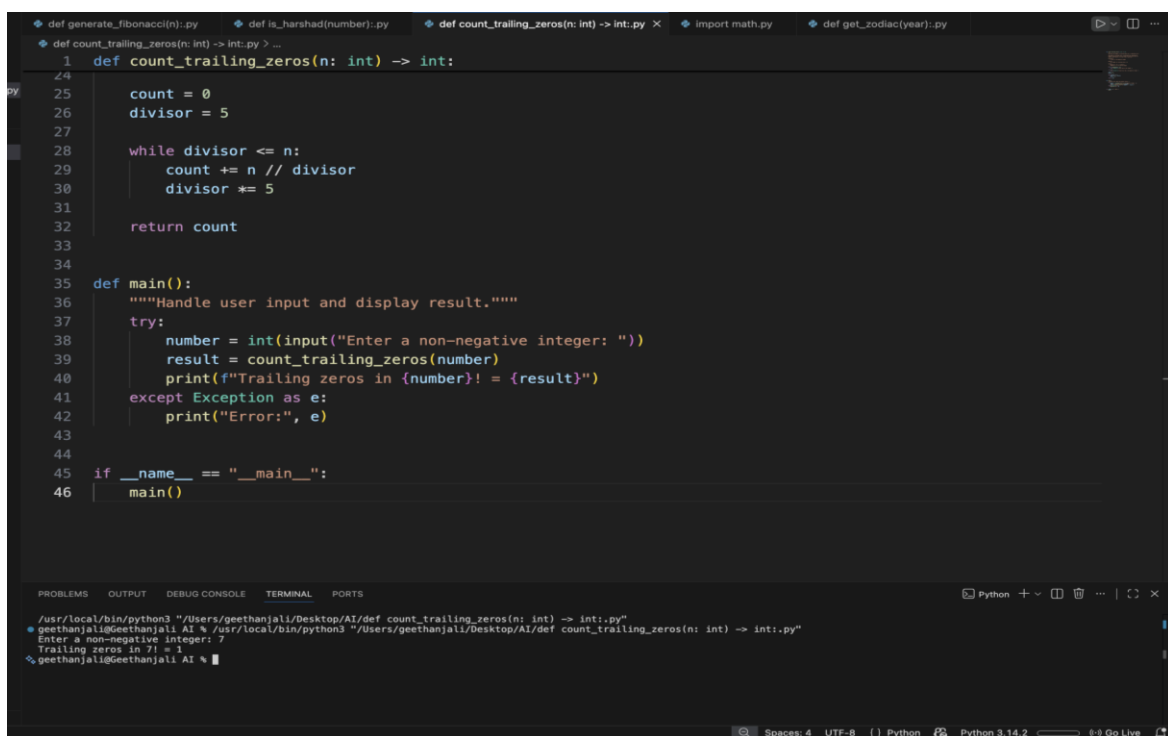
You must refactor the code to improve efficiency, readability, and maintainability.

Factorial Trailing Zeros (Unstructured Version)

```
n = int(input("Enter a number: ")) fact = 1 i = 1 while i <= n: fact = fact * i i = i + 1 count = 0 while fact % 10 == 0: count = count + 1 fact = fact // 10 print("Trailing zeros:", count)
```

You must:

1. Create a reusable function: `count_trailing_zeros(n)`
2. The function must:
 - o Accept a non-negative integer n .
 - o Return the number of trailing zeros in $n!$.
3. Do NOT compute the full factorial.
4. Use an optimized mathematical approach (count multiples of 5).
5. Add proper docstrings.
6. Separate user interaction from core logic.
7. Handle edge cases (e.g., negative numbers, zero).



```
def count_trailing_zeros(n: int) -> int:
    """
    Calculate the number of trailing zeros in n! using an optimized
    mathematical approach (counting multiples of 5).
    """
    count = 0
    divisor = 5
    while divisor <= n:
        count += n // divisor
        divisor *= 5
    return count

def main():
    """Handle user input and display result."""
    try:
        number = int(input("Enter a non-negative integer: "))
        result = count_trailing_zeros(number)
        print(f"Trailing zeros in {number}! = {result}")
    except Exception as e:
        print("Error:", e)

if __name__ == "__main__":
    main()
```

The screenshot shows a Python IDE with the following code in the editor:

```
def count_trailing_zeros(n: int) -> int:
    """
    Calculate the number of trailing zeros in n! using an optimized
    mathematical approach (counting multiples of 5).
    """
    count = 0
    divisor = 5
    while divisor <= n:
        count += n // divisor
        divisor *= 5
    return count

def main():
    """Handle user input and display result."""
    try:
        number = int(input("Enter a non-negative integer: "))
        result = count_trailing_zeros(number)
        print(f"Trailing zeros in {number}! = {result}")
    except Exception as e:
        print("Error:", e)

if __name__ == "__main__":
    main()
```

The terminal output shows the execution of the program:

```
Enter a non-negative integer: 7
Trailing zeros in 7! = 1
```

Task 13 (Collatz Sequence Generator – Test Case Design)

- Function: Generate Collatz sequence until reaching 1.
- Test Cases to Design:
- Normal: $6 \rightarrow [6, 3, 10, 5, 16, 8, 4, 2, 1]$
- Edge: $1 \rightarrow [1]$
- Negative: -5
- Large: 27 (well-known long sequence)
- Requirement: Validate correctness with pytest.

Explanation:

We need to write a function that:

- Takes an integer n as input.
- Generates the Collatz sequence (also called the $3n+1$ sequence).
- The rules are:
 - If n is even $\rightarrow \text{next} = n / 2$.
 - If n is odd $\rightarrow \text{next} = 3n + 1$.
- Repeat until we reach 1.
- Return the full sequence as a list.

Example

Input: 6

Steps:

- 6 (even $\rightarrow 6/2 = 3$)
- 3 (odd $\rightarrow 3*3+1 = 10$)
- 10 (even $\rightarrow 10/2 = 5$)
- 5 (odd $\rightarrow 3*5+1 = 16$)
- 16 (even $\rightarrow 16/2 = 8$)
- 8 (even $\rightarrow 8/2 = 4$)
- 4 (even $\rightarrow 4/2 = 2$)
- 2 (even $\rightarrow 2/2 = 1$)

Output:

[6, 3, 10, 5, 16, 8, 4, 2, 1]

```
def generate_fibonacci(n):
def is_harshad(number):
def count_trailing_zeros(n: int) -> int:
def test_generate_collatz_sequence():
import math.py

1 def test_generate_collatz_sequence():
2     assert generate_collatz_sequence(6) == [6, 3, 10, 5, 16, 8, 4, 2, 1], "Test case 1 failed"
3     assert generate_collatz_sequence(27) == [27, 82, 41, 124, 62, 31, 94, 47, 142, 71, 214, 107, 322, 161, 484,
4     assert generate_collatz_sequence(1) == [1], "Test case 3 failed"
5     try:
6         generate_collatz_sequence(-5)
7         assert False, "Test case 4 failed: ValueError not raised for negative input"
8     except ValueError:
9         pass
10    try:
11        generate_collatz_sequence(0)
12        assert False, "Test case 5 failed: ValueError not raised for zero input"
13    except ValueError:
14        pass
15    print("All test cases passed!")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python Python 3.14.2 6-8 Go Live

/usr/local/bin/python3 "/Users/geethanjali/Desktop/AI/def test_generate_collatz_sequence():.py"
geethanjali@Geethanjali AI % /usr/local/bin/python3 "/Users/geethanjali/Desktop/AI/def test_generate_collatz_sequence():.py"
All test cases passed!
geethanjali@Geethanjali AI %
```

Task 14 (Lucas Number Sequence – Test Case Design)

- Function: Generate Lucas sequence up to n terms.
(Starts with 2,1, then $F_n = F_{n-1} + F_{n-2}$)
- Test Cases to Design:
- Normal: $5 \rightarrow [2, 1, 3, 4, 7]$
- Edge: $1 \rightarrow [2]$
- Negative: $-5 \rightarrow \text{Error}$
- Large: 10 (last element = 76).
- Requirement: Validate correctness with pytest.

```
def count_vowels_consonants(s: str) -> tuple:
1 def count_vowels_consonants(s: str) -> tuple:
2     """Counts the number of vowels and consonants in a given string.
3     Parameters:
4     s (str): The input string to analyze
5     Returns:
6     tuple: A tuple containing the count of vowels and consonants in the format (vowels, consonants)"""
7     vowels = 'aeiouAEIOU'
8     vowel_count = sum(1 for char in s if char in vowels)
9     consonant_count = sum(1 for char in s if char.isalpha() and char not in vowels)
10    return (vowel_count, consonant_count)
11 def test_count_vowels_consonants():
12     assert count_vowels_consonants("hello") == (2, 3), "Test case 1 failed"
13     assert count_vowels_consonants("") == (0, 0), "Test case 2 failed"
14     assert count_vowels_consonants("aeiou") == (5, 0), "Test case 3 failed"
15     assert count_vowels_consonants("This is a long text with multiple vowels and consonants.") == (16, 29), "Test case 4 failed"
16    print("All test cases passed!")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
python Python 3.14.2 6-8 Go Live

/usr/local/bin/python3 "/Users/geethanjali/Desktop/AI/def count_vowels_consonants(s: str) -> tuple.py"
geethanjali@Geethanjali AI % /usr/local/bin/python3 "/Users/geethanjali/Desktop/AI/def count_vowels_consonants(s: str) -> tuple.py"
All test cases passed!
geethanjali@Geethanjali AI %
```

Task 15 (Vowel & Consonant Counter – Test Case Design)

- Function: Count vowels and consonants in string.

- Test Cases to Design:
- Normal: "hello" $\rightarrow$ (2,3)
- Edge: "" $\rightarrow$ (0,0)
- Only vowels: "aeiou" $\rightarrow$ (5,0)

Large: Long text

- Requirement: Validate correctness with pytest.

```

def generate_lucas_sequence(n: int) -> list:
    """Generates the Lucas sequence up to n terms.
    The Lucas sequence starts with 2, 1, and each subsequent term is the sum of the two preceding terms.
    Parameters:
    n (int): The number of terms to generate in the Lucas sequence
    Returns:
    list: A list containing the first n terms of the Lucas sequence
    """
    if n <= 0:
        raise ValueError("Input must be a positive integer.")
    if n == 1:
        return [2]
    lucas_sequence = [2, 1]
    for i in range(2, n):
        lucas_sequence.append(lucas_sequence[i-1] + lucas_sequence[i-2])
    return lucas_sequence

def test_generate_lucas_sequence():
    assert generate_lucas_sequence(5) == [2, 1, 3, 4, 7], "Test case 1 failed"
    assert generate_lucas_sequence(1) == [2], "Test case 2 failed"
    try:
        generate_lucas_sequence(-5)
        assert False, "Test case 3 failed: ValueError not raised for negative input"
    except ValueError:
        pass
    assert generate_lucas_sequence(10) == [2, 1, 3, 4, 7, 11, 18, 29, 47, 76], "Test case 4 failed"
  
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

/usr/local/bin/python3 /Users/geethanjali/Desktop/AI/def generate_lucas_sequence(n: int) -> l.py
geethanjali@geethanjali AI % /usr/local/bin/python3 /Users/geethanjali/Desktop/AI/def generate_lucas_sequence(n: int) -> l.py
geethanjali@geethanjali AI %
  
```

Spaces: 4 UTF-8 Python Python 3.14.2 Go Live